

CHRONO RIFT

A Multi-Process Turn-Based Combat Game

Operating Systems Semester Project

Submitted by

Muhammad Hassaan ul Mustafa

24i-0717

Muhammad Taha

24i-0584

Department of Computer Science

FAST-NUCES Islamabad

1. Introduction

Chrono Rift is a turn-based combat game implemented as a distributed three-process system on Linux. The project demonstrates the practical application of operating-system concepts taught in the course: process creation, POSIX shared memory, pthread-based synchronization, signal handling, and condition-variable coordination across process boundaries.

The game pits one to four player-controlled heroes against a swarm of NPC enemies. Each character is backed by its own thread, all entities share a single block of POSIX shared memory, and turn-taking is coordinated via mutexes, condition variables, and POSIX timers. The project intentionally avoids any higher-level concurrency wrappers; every synchronization primitive used is a low-level POSIX or pthread call.

1.1 Project Goals

- Implement a fully working three-process architecture using only POSIX shared memory (no pipes, no FIFOs, no sockets).
- Spawn one dedicated pthread per player and per NPC and synchronize them through a single shared GameState structure.
- Demonstrate signal-based suspension and resumption of an entire process (SIGSTOP / SIGCONT) for the Ultimate Ability and Stun mechanics.
- Use POSIX interval timers to enforce time-bounded turns and stun durations.
- Provide a polished SFML graphical front-end with an inventory grid, animations, and live battle log.
- Build cleanly inside the provided Docker container with zero warnings.

2. System Architecture

The game runs as three independent Linux processes that communicate exclusively through one shared-memory segment named `/chrono_shm`. There are no pipes, sockets, or files used for inter-process communication.

2.1 The Three Processes

Arbiter (arbiter_app)	The owner. Creates the shared-memory segment, initializes the cross-process mutex and condition variable, runs the SFML render loop and main game scheduler, owns the POSIX timers, and tears the segment down on exit.
HIP (hip_app)	Hero Interaction Process. Spawns one pthread per human-controlled hero. Each thread waits on the shared condition variable until it is the hero's turn and a player action has been confirmed, then mutates inventory / HP / stamina under the mutex.
ASP (asp_app)	Adversary Simulation Process. Spawns one pthread per NPC. Each thread waits on the same condition variable, then chooses Strike / Skip / Ultimate based on a seeded RNG and applies damage to the heroes.

2.2 Shared Memory Layout

All inter-process state lives in one struct, GameState, mapped into all three processes via mmap.

Key fields include:

- `pthread_mutex_t mutex` - the global lock guarding every read/write to GameState.
- `pthread_cond_t turn_cv` - condition variable used to wake hero/NPC threads when a turn change occurs.
- `Character players[4]`, `Character npcs[9]` - the player and NPC arrays with HP, stamina, speed, inventory, and storage.
- `active_id`, `is_player_turn`, `action_triggered` - the turn-state tuple modified atomically under the mutex.
- Artifact registry, pickup-prompt state, ultimate / stun / animation channels.
- Process IDs (`arbiter_pid`, `hip_pid`, `asp_pid`) so processes can send signals to one another.

The mutex is initialized with `PTHREAD_PROCESS_SHARED` so it works across process boundaries; the same applies to the condition variable. This is critical: without the `PTHREAD_PROCESS_SHARED` attribute, a pthread mutex placed in shared memory has undefined behavior when locked from a different process.

3. Operating-System Concepts Used

This section maps every OS concept covered in the course to its concrete use in the codebase.

3.1 Processes

- fork-and-exec model: the user launches three executables manually; each is a top-level process with its own PID.
- Each process publishes its PID into the shared segment so peers can target it with kill().
- Cross-process signal delivery (SIGSTOP, SIGCONT, SIGTERM, SIGALRM, SIGRTMIN) drives time-bounded mechanics.

3.2 Threads (pthreads)

- pthread_create spawns one dedicated thread per hero in HIP and one per NPC in ASP.
- pthread_join on shutdown so resources are cleaned up cleanly.
- Background utility threads in the Arbiter: render-snapshot thread, deadlock monitor, ultimate monitor, stun monitor.

3.3 Synchronization

Mechanism	Purpose	Where in code
pthread_mutex_t	Mutual exclusion over GameState	shared_data.h (struct field), arbiter main() (init)
pthread_cond_t (process-shared)	Wake hero/NPC threads on turn change	shared_data.h, arbiter main() (init), asp/hip behavior threads
pthread_mutexattr_setpshared	Mark mutex as cross-process safe	arbiter main() at SHM init

Mechanism	Purpose	Where in code
pthread_condattr_setpshared	Mark condvar as cross-process safe	arbiter main() at SHM init
POSIX SHM (shm_open + mmap)	Mapped GameState into all three processes	arbiter / hip / asp main()

3.4 Shared Memory

```
// arbiter.cpp - the only process that creates the segment
shm_unlink(SHM_NAME);           // clean any stale segment
int fd = shm_open(SHM_NAME, O_CREAT|O_RDWR, 0666);
ftruncate(fd, sizeof(GameState));
GameState* state = mmap(0, sizeof(GameState),
                        PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
```

HIP and ASP open the same segment with shm_open(O_RDWR) - they never create or destroy it, eliminating the risk of one of them tearing the segment down while another is using it.

3.5 Signals

Signal	Purpose	Sender to receiver
SIGSTOP / SIGCONT	Suspend/resume entire ASP process during Ultimate Ability or NPC stun	Arbiter to ASP

Signal	Purpose	Sender to receiver
SIGALRM	Stun timer expiry (3 s)	Kernel timer to Arbiter
SIGRTMIN	Ultimate window expiry (10 s)	Kernel timer to Arbiter
SIGTERM	Player-initiated quit	HIP to Arbiter

All signal handlers are installed with `sigaction` (POSIX-portable) rather than the older `signal()` call. `SA_RESTART` is set so interrupted system calls auto-resume; `SA_RESETHAND` is intentionally not set so the handler stays installed for repeated events.

3.6 POSIX Timers

Two independent POSIX interval timers (`timer_create` with `CLOCK_MONOTONIC`) drive time-bounded gameplay events:

- `g_stun_timer` fires `SIGALRM` after 3 seconds for the stun-recovery mechanic.
- `g_ultimate_timer` fires `SIGRTMIN` after 10 seconds for the Ultimate Ability suspension window.

Each timer has its own signal and its own handler, so arming one never overwrites the countdown of the other. This replaces an earlier `single-alarm()` design where a second `alarm()` call would silently overwrite the first.

3.7 Critical-Section Discipline

Every read or write of GameState fields (with the documented exceptions of last_input_ms and a few volatile sig_atomic_t flags used inside signal handlers) is performed inside pthread_mutex_lock / pthread_mutex_unlock blocks. The codebase uses three disciplined patterns:

- Lock-test-act-unlock for short critical sections.
- Lock, take a local snapshot, unlock, then act on the snapshot - used in the renderer to avoid holding the lock during slow SFML draw calls.
- Lock, wait on condition variable, re-test predicate after wake - the textbook pattern for condition-variable use.

4. Game Mechanics Implemented

4.1 Turn Scheduling

Each character has a stamina bar and a per-second speed value. The Arbiter increments stamina once per wall-clock second under the mutex. A hero acts when stamina reaches 100; an NPC acts when stamina reaches 150. On a tie, players are scheduled first (player priority). The scheduler then broadcasts on turn_cv to wake the relevant behavior thread.

4.2 NPC Turn Timeout (3-second budget)

If ASP fails to submit a move within 3 wall-clock seconds, the Arbiter force-skips the NPC: stamina drops to 75 (50% of max), active_id resets to -1, and an npc_turn_aborted flag is raised

so ASP can log the event cleanly. The ASP behavior thread additionally re-verifies `active_id` under the mutex after waking, so a stale write into a no-longer-current turn is impossible.

4.3 Inventory and Storage

Each hero has a 20-slot primary inventory and a 40-slot long-term storage. Weapons of slot size N occupy N contiguous slots. The first-fit allocator handles fragmentation; an evict-and-place fallback is invoked when contiguous space is unavailable. The Solar Core and Lunar Blade each occupy 10 slots, and the code enforces the spec's hard constraint that a hero holding both has zero free space for a third weapon.

4.4 Weapon Drops and Pickups

On enemy death the code rolls a 70% drop chance. If a hero already holds Solar+Lunar (full inventory), the drop is funneled directly to a random alive enemy without raising a futile pickup prompt. Otherwise the active hero is shown a Yes/No prompt; declining gives the weapon to a random NPC. Legendary weapons (Solar Core, Lunar Blade, Eclipse Relic) are tracked in a global artifact registry so only one of each exists in the world at any time.

4.5 Eclipse Relic (surprise weapon)

The Eclipse Relic does not exist at game start. After a randomized 2-5 enemy kills, it appears in the world and the next character on the battlefield is offered the chance to pick it up. The relic is fully integrated with the artifact registry: if the holder dies, it is freed and may re-enter the drop pool.

4.6 Ultimate Ability

A hero holding both Solar Core and Lunar Blade unlocks the Ultimate menu option. Activating it does 200 damage to every alive NPC and triggers a 10-second window during which ASP is fully suspended via SIGSTOP. After 10 seconds, the kernel-driven ultimate timer fires SIGRTMIN, the handler sends SIGCONT to ASP, and gameplay resumes.

4.7 Stun Mechanic

Striking an NPC with a stun-flagged weapon (Solar Core, Lunar Blade, Iron Halberd, Thunderstaff, Eclipse Relic) freezes that NPC for exactly 3 seconds via the same SIGSTOP / SIGALRM mechanism, but using the dedicated stun timer rather than the ultimate timer.

4.8 Deadlock Detection

When a hero holds one legendary and an NPC holds the other, the Arbiter's deadlock-monitor thread detects the circular wait via the (holds, wants) artifact graph and forcibly releases one party's grip, flashing a banner on screen. This implements the classical wait-for-graph cycle detection.

4.9 Multiplayer (1-4 Heroes)

The game supports solo (1 player) and multiplayer (2 players) modes. In multiplayer, both players share one keyboard. Inputs are debounced via a 120-millisecond window so a held key from one player cannot leak into the other player's turn. The action panel never moves between turns: a single unified panel is drawn at a fixed position with a 'PLAYER N's TURN' label that updates as turns change.

5. Build and Run Instructions

5.1 Dependencies

- Ubuntu 22.04 (the supplied Docker base) or 24.04.
- g++ supporting C++17.
- libsfml-dev (graphics, window, audio, network, system).
- librt (linked for POSIX timers and clock_gettime).

5.2 Building

```
$ cd ChronoRift  
  
$ make  
  
g++ -Wall -Wextra -std=c++17 -pthread -I. arbiter/*.cpp -o arbiter_app ...  
g++ -Wall -Wextra -std=c++17 -pthread -I. hip/*.cpp -o hip_app ...  
g++ -Wall -Wextra -std=c++17 -pthread -I. asp/*.cpp -o asp_app ...  
  
Build complete.
```

The build is clean: zero errors, zero warnings.

5.3 Running

The Arbiter must be started first (it owns the shared-memory segment); HIP and ASP can be launched in either order afterwards.

```
Terminal 1: $ ./arbiter_app  
Terminal 2: $ ./hip_app  
Terminal 3: $ ./asp_app
```

On the welcome screen, press Enter, type a player name, choose Solo or Multiplayer, optionally enter a second player name, then pick the party size with Up/Down arrows and confirm with Enter. The battle screen appears and the threads in HIP and ASP wake up.

5.4 Docker

A Dockerfile is provided. From the project root:

```
$ docker build -t chronorift .  
$ docker run -it --rm chronorift
```

The image installs SFML, copies the source tree, and runs make automatically, dropping the user into a bash shell where ./arbiter_app etc. can be launched.

6. Highlights of the Implementation

6.1 Cross-Process Condition Variable

Earlier iterations of the project polled the turn state in a 50-millisecond busy loop. The current implementation uses a process-shared pthread_cond_t initialized with PTHREAD_PROCESS_SHARED. Behavior threads block on pthread_cond_timedwait with a 200-millisecond safety-net timeout, and any state-changing thread broadcasts on the variable before releasing the mutex. This is a strictly event-driven design while still being robust against missed broadcasts.

6.2 Two Independent POSIX Timers

Stun (3 s) and Ultimate (10 s) each have their own timer_create handle, fire distinct signals (SIGALRM and SIGRTMIN), and have distinct handlers. Arming one timer cannot alter the countdown of the other - a property that the older single-alarm() design lacked.

6.3 Reproducible Randomness

All three processes call srand(get_roll_seed()) at startup, where get_roll_seed reads ROLL_NUMBER from the environment (defaulting to 240584). NPC behavior, drop rolls, and the Eclipse Relic unlock threshold are therefore identical across runs given the same seed - useful for debugging and grading.

6.4 Render Isolation

SFML requires draw calls to happen on the window-creating thread. A separate render-prep thread takes a brief snapshot of GameState under the mutex and copies the visible fields to a thread-local RenderSnapshot. The main thread then draws from the snapshot only - no SFML call ever holds the global mutex, so there are no dropped frames during contended turns.

6.5 Input Debouncing for Shared-Keyboard Multiplayer

A single last_input_ms timestamp is recorded for every accepted keypress. Subsequent keypresses arriving within INPUT_DEBOUNCE_MS (120 ms) are silently dropped. This prevents one player's held keystroke from leaking into the other player's turn while leaving normal menu navigation responsive.

7. Testing and Verification

7.1 Compile-time

- Builds cleanly with -Wall -Wextra on g++ 13.3.0 and libsFML 2.6.1.
- Zero errors, zero warnings.

7.2 Functional

- Solo mode (1 to 4 heroes) confirmed: turn handoff between hero and NPC works, drops fire at expected rates, win at 10 kills, defeat when all heroes die.
- Multiplayer mode (2 heroes) confirmed: shared keyboard, debounce prevents cross-talk, turn label updates correctly as turns change.
- Ultimate Ability: ASP correctly suspends for 10 seconds and resumes.
- Stun Mechanic: targeted NPC freezes for 3 seconds, then resumes from the same stamina value (signal-based suspension preserves state exactly).
- NPC turn timeout: when ASP is forcibly killed mid-turn, Arbiter recovers within 3 seconds and continues the game.
- Deadlock detection: contrived test where hero takes Solar Core, declines Lunar Blade (NPC takes it), then attempts to pick up Lunar Blade triggers the cycle banner.

7.3 Restriction Compliance

No <code>std::thread</code> / <code>std::mutex</code> / <code>std::atomic</code>	Verified by source grep - no banned STL concurrency headers included.
---	---

No pipes or FIFOs	Verified by source grep - no pipe / pipe2 / mkfifo / popen calls.
One thread per player	hip.cpp main() spawns num_players threads via pthread_create.
One thread per NPC	asp.cpp main() spawns num_npcs threads via pthread_create.
Three separate processes	arbiter_app, hip_app, asp_app are independent executables.
Docker buildable	Dockerfile installs dependencies and runs make as part of the image build.

8. Conclusion

Chrono Rift demonstrates the synthesis of low-level OS primitives - processes, threads, shared memory, mutexes, condition variables, signals, and POSIX timers - into a complete, polished application. Every coordination mechanism in the game maps to a concept covered in the Operating Systems course, and every concept is used the way it is taught: pthread mutexes for mutual exclusion, condition variables for event-driven wake-ups, shared memory for IPC, signals for asynchronous notification, and POSIX timers for time-bounded events. No higher-level wrappers are used.

Implementing the project gave us first-hand experience with the subtleties that classroom examples often gloss over: the necessity of PTHREAD_PROCESS_SHARED on both mutexes and condition variables placed in shared memory; the difference between a single shared alarm()

and two independent POSIX timers; the trap of busy-polling versus the cleaner event-driven design unlocked by condition variables; and the importance of disciplined critical sections when several processes mutate the same struct.

The result is a game that is simultaneously fun to play, defensibly correct, and a faithful illustration of the core operating-system mechanics.